

GitHub Actions — Reference Guide

GitLab se GitHub migration ke liye | Hinglish | pytest + Selenium + S3 context

- | | |
|--------------------------------------|-----------------------------------|
| 1. GitLab → GitHub Translation Table | 9. Artifacts, Cache, Outputs |
| 2. Push ke baad ka Lifecycle | 10. Matrix Strategy |
| 3. Workflow File Structure | 11. Logs kaise dekhein |
| 4. name / uses / run | 12. Locally chalana (act, gh CLI) |
| 5. Triggers (on:) | 13. Reusable Workflows |
| 6. Jobs ke keys | 14. Expression Syntax |
| 7. Steps ke keys | 15. Common Gotchas |
| 8. Secrets, Variables, Permissions | 16. Complete Working Example |

1. GitLab → GitHub Translation Table

Logic wahi rehta hai, sirf keywords badalte hain.

GitLab CI	GitHub Actions
<code>.gitlab-ci.yml</code> (root me)	<code>.github/workflows/name.yml</code>
<code>stages:</code> + <code>job:</code>	<code>jobs:</code> + <code>steps:</code>
<code>script:</code>	<code>run:</code>
<code>image: python:3.11</code>	<code>runs-on:</code> + <code>actions/setup-python</code>
<code>only:</code> / <code>rules:</code>	<code>on: push: branches:</code>
Pipeline Schedules (UI setting)	<code>on: schedule: - cron:</code> (code me)
<code>artifacts: paths:</code>	<code>actions/upload-artifact@v4</code>
<code>when: always</code>	<code>if: always()</code>
<code>cache:</code>	<code>cache: pip ya actions/cache@v4</code>
Settings → CI/CD → Variables (masked)	Settings → Secrets and variables → Secrets
"Run pipeline" button	<code>workflow_dispatch:</code>
GitLab Runner	GitHub-hosted / self-hosted runner
<code>needs:</code>	<code>needs:</code> (same!)
<code>parallel: matrix:</code>	<code>strategy: matrix:</code>

Sabse bada difference: GitLab me schedule ek UI setting thi. GitHub me schedule **code ke andar** likha jaata hai, aur woh sirf default branch (`main`) se chalta hai.

2. Push ke baad ka Lifecycle

#	Kya hota hai	Kaun karta hai
1	Tum <code>git push</code> karte ho	Tum
2	GitHub event payload banata hai (branch, SHA, author, changed files)	GitHub
3	<code>.github/workflows/</code> ki saari files scan hoti hain	GitHub
4	Jis file ka <code>on:</code> match kare, uska workflow run banta hai	GitHub
5	Har job ke liye fresh VM boot hoti hai (~10-30 sec)	GitHub
6	Runner agent start, env vars set, <code>GITHUB_TOKEN</code> mint, secrets inject (memory me, disk pe nahi)	Runner
7	Steps ek-ek karke execute — har <code>run:</code> ek temp bash script banti hai	Runner
8	Logs real-time stream hote hain, exit code report hota hai	Runner
9	VM destroy ho jaati hai — sab kuch gone	GitHub

Runner me kya milta hai (`ubuntu-latest`)

Cheez	Detail
Hardware	4 vCPU, 16 GB RAM, 14 GB SSD
OS	Ubuntu 24.04
Pre-installed	Python 3.9-3.13, Node, Docker, Chrome, Firefox, git, aws-cli, jq
Access	Passwordless <code>sudo</code>
Working dir	<code>/home/runner/work/<repo>/<repo></code>

Steps ke beech kya persist hota hai

Cheez	Persist?
Files on disk	Haan
Installed packages	Haan
Working directory (<code>cd</code>)	Nahi — har step fresh shell
Shell variables (<code>export</code>)	Nahi
Env via <code>\$GITHUB_ENV</code>	Haan

```
- run: export MY_VAR=hello           # agle step me gayab
- run: echo "MY_VAR=hello" >> $GITHUB_ENV # persist hoga
- run: echo $MY_VAR                  # hello
```

Step 9 sabse important hai. Job khatam → VM permanently delete. No SSH, no recovery. Jo bachana hai woh artifact ya S3 me bhejo — job khatam hone se pehle.

3. Workflow File Structure

```

Workflow file (.yaml)
|
|-- name:           <- workflow ka naam
|-- on:             <- TRIGGER (mandatory)
|-- permissions:    <- token ki power
|-- env:            <- sab jobs ke liye variables
|-- defaults:       <- default shell / working-directory
|-- concurrency:    <- purane run cancel karo
|
|-- jobs:           <- JOBS (mandatory, parallel by default)
    |-- job_id:
        |-- name:
        |-- runs-on: <- kaunsi machine
        |-- needs:   <- kis job ke baad
        |-- if:      <- condition
        |-- timeout-minutes:
        |-- strategy: <- matrix
        |-- env:
        |-- outputs:  <- agle job ko data
        |
        |-- steps:    <- STEPS (hamesha sequential)
            |-- - name:
            |   uses:   <- ready-made action
            |   with:   <- action ke inputs
            |
            |-- - name:
            |   run:    <- shell command
            |   env:
            |   if:

```

Golden rule: Jobs **parallel** chalte hain (jab tak **needs:** na ho). Steps hamesha **sequential**.
Sif **on:** aur **jobs:** compulsory hain. Baaki sab optional.

4. name / uses / run

Keyword	Analogy (recipe)	Kaam
name	Step ka title — "Onion bhuno"	Sif label. Kuch karta nahi, log me dikhta hai
uses	"Ready-made masala packet daalo"	Marketplace ka bana-banaya block
run	"5 min medium flame pe pakao"	Apni shell command

	uses:	run:
Kya	Ready-made action	Shell command
Kahan se	Marketplace / koi repo	Tumhara dimaag
Input kaise	with: block	Command me ya env:
Version	@v5 pin karna padta hai	N/A

Ek step me uses YA run — dono kabhi nahi. Dono chahiye to do alag steps banao.

Kab kya use karo

Kaam	Tool
Repo clone	uses: actions/checkout@v5
Python / Node install	uses: actions/setup-python@v5
pip install / npm install	run:
Tests chalana	run:
Artifact upload	uses: actions/upload-artifact@v4
AWS / Azure / GCP login	uses: (official action)
aws s3 cp	run: (CLI pre-installed)
Custom bash logic	run:

Heuristic: API calls / retry logic / credentials handling ho → **uses:** dhundo. Simple terminal command ho → **run:** .

5. Triggers (on:)

```
on:
  push:
    branches: [main, develop]
    paths: ['src/**', 'tests/**'] # sirf inme change ho tab
    paths-ignore: ['**/*.md']    # docs change pe skip

  pull_request:
    types: [opened, synchronize, reopened]

  schedule:
    - cron: "30 2 * * *"          # UTC ONLY = 8:00 AM IST

  workflow_dispatch:              # manual button
    inputs:
      test_suite:
        description: 'Which suite to run'
        type: choice
        options: [smoke, regression, all]
        default: smoke
        required: true

  workflow_call:                  # dusri workflow call kar sake
```

Trigger	Kab use karo
<code>push</code>	Har commit pe validation
<code>pull_request</code>	Merge se pehle gate
<code>schedule</code>	Nightly regression, cleanup
<code>workflow_dispatch</code>	Manual / on-demand run
<code>workflow_call</code>	Reusable workflow (DRY)

Cron ke 3 rules

1. **Cron hamesha UTC me hai.** IST se 5:30 ghante peeche. 9 AM IST chahiye → `30 3 * * *`
2. **Schedule sirf default branch se chalta hai.** Feature branch pe cron daala to kabhi nahi chalega.
3. **60 din tak repo me commit na ho to GitHub schedule silently disable kar deta hai.** Email aata hai, log miss kar dete hain.

`workflow_dispatch`: **hamesha** add karo. Bina iske test karne ke liye dummy commit karna padega.

6. Jobs ke keys

```
jobs:
  test:
    # job ID
    name: Run Selenium Suite    # display name
    runs-on: ubuntu-latest      # REQUIRED
    timeout-minutes: 30
    if: github.ref == 'refs/heads/main'

    strategy:
      fail-fast: false
      matrix:
        python: ["3.11", "3.12"]

    env:
      HEADLESS: "true"

    outputs:
      report: ${ steps.run.outputs.path }

    steps: [...]

  deploy:
    needs: test                # test ke baad chalega
    runs-on: ubuntu-latest
    steps: [...]
```

Key	Kaam
<code>runs-on</code>	Required. VM type
<code>needs</code>	Dependency — sequential banata hai
<code>if</code>	Job chale ya nahi
<code>timeout-minutes</code>	Default 6 hours! Hamesha set karo
<code>strategy.matrix</code>	Ek job → multiple parallel variants
<code>outputs</code>	Agle job ko chhota data bhejo
<code>container</code>	Custom Docker image me chalao
<code>services</code>	Sidecar containers (Postgres, Redis)

Runner options

Runner	Kya hai
<code>ubuntu-latest</code>	Default choice — sasta, sab pre-installed
<code>windows-latest</code>	Windows Server
<code>macos-latest</code>	Mac — ~10x costly, mostly iOS ke liye
<code>[self-hosted, linux]</code>	Tumhari apni machine — VPN / internal network access ke liye

7. Steps ke keys

```
steps:
  - name: Checkout
    uses: actions/checkout@v5
    with:
      fetch-depth: 0

  - name: Run tests
    id: run                    # outputs reference ke liye
    run: |
      mkdir -p reports
      pytest tests/ -v
    env:
      APP_PASSWORD: ${ secrets.APP_PASSWORD }
    working-directory: ./src
    continue-on-error: false
```

Key	Kaam
<code>name</code>	Log me label
<code>id</code>	Output reference ke liye
<code>uses</code>	Ready-made action
<code>with</code>	<code>uses</code> ke inputs (<code>run</code> ke saath kaam nahi karta)
<code>run</code>	Shell command
<code>shell</code>	<code>bash</code> / <code>pwsh</code> / <code>python</code>
<code>env</code>	Sirf is step ke variables
<code>if</code>	Condition
<code>working-directory</code>	<code>cd</code> kar ke chalao
<code>continue-on-error</code>	Fail ho to bhi aage badho

Pehla step hamesha `checkout` hona chahiye. Runner pe by default tumhara code hota hi nahi — VM khali hai. Ye bhoolna GitHub Actions ki #1 beginner mistake hai.

8. Secrets, Variables, Permissions

Repo → Settings → Secrets and variables → Actions

Type	Syntax	Log me dikhta?	Kya rakho
Secret	<code>\${{ secrets.API_KEY }}</code>	Nahi — <code>***</code>	Password, token, key
Variable	<code>\${{ vars.BASE_URL }}</code>	Haan	URL, region, config
Env	<code>\${{ env.MY_VAR }}</code>	Haan	Workflow-internal value

Env ke 3 levels

```
env:
    # workflow-level (sab jobs)
    AWS_REGION: us-east-1

jobs:
  test:
    env:
      # job-level (sab steps)
      HEADLESS: "true"
    steps:
      - run: pytest
        env:
          # step-level (sirf ye step)
          APP_PASSWORD: ${{ secrets.APP_PASSWORD }}
```

Permissions

Permission	Kya milta hai
<code>contents: read</code>	Sirf repo clone / read
<code>contents: write</code>	Code push, tag, release
<code>id-token: write</code>	OIDC — AWS / Azure / Vault login
<code>pull-requests: write</code>	PR pe comment

OIDC vs stored keys.
Purana tarika (chaabi): AWS ki permanent key GitHub Secrets me. Leak hui to game over, rotate karna manual headache.
OIDC (visitor pass): GitHub har run pe AWS ko bolta hai "main is repo ka run hoon" → AWS 15-min ka temporary pass deta hai. Koi key store hi nahi hoti.

Gotcha: Secrets fork se aaye PRs me available nahi hote. Ye security feature hai, bug nahi.

9. Conditions, Artifacts, Cache, Outputs

Conditions

Kya chahiye	Syntax
Fail hone pe bhi chale	<code>if: always()</code>
Sirf fail hone pe	<code>if: failure()</code>
Sirf success pe (default)	<code>if: success()</code>
Sirf specific trigger pe	<code>if: github.event_name == 'schedule'</code>
Sirf main branch pe	<code>if: github.ref == 'refs/heads/main'</code>
Fail ho to bhi job green rahe	<code>continue-on-error: true</code>

`if: always()` **Selenium pipeline ka hero hai.** Bina iske: pytest fail → job abort → report upload skip. Aur report exactly usi din chahiye hoti hai jab tests fail hote hain.

Data movement — 3 tareeke

Tarika	Scope	Use case
Artifacts	Run ke baad bhi (retention days)	Reports, screenshots, builds
Cache	Runs ke beech reuse	pip / npm dependencies
Outputs	Job → Job, chhota data	Version number, status flag

```
# Artifact
- uses: actions/upload-artifact@v4
  if: always()
  with:
    name: reports-${{ github.run_number }}
    path: reports/
    retention-days: 30

# Cache (built-in, simplest)
- uses: actions/setup-python@v5
  with:
    python-version: "3.11"
    cache: pip
    cache-dependency-path: requirements.txt

# Cache (manual)
- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: pip-${{ hashFiles('requirements.txt') }}
```

```
# Outputs: Job A -> Job B
jobs:
  build:
    outputs:
      version: ${ steps.get.outputs.ver }
    steps:
      - id: get
        run: echo "ver=1.2.3" >> $GITHUB_OUTPUT
  deploy:
    needs: build
    steps:
      - run: echo ${ needs.build.outputs.version }
```

`hashFiles()` ka matlab: jab tak `requirements.txt` na badle, cache reuse hoga. Install time 2 min se 15 sec ho jaata hai.

10. Matrix Strategy

```
jobs:
  test:
    strategy:
      fail-fast: false          # ek fail ho to baaki cancel na ho
      matrix:
        python: ["3.10", "3.11"]
        browser: [chrome, firefox]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python }
```

Ye **4 parallel jobs** banayega (2 × 2). Cross-browser Selenium testing ke liye perfect.

11. Logs kaise dekhein

Web UI

Repo → Actions tab → workflow name → run click → job click → steps expand

Chahiye	Kaise
Poore run ka log download	Gear icon → Download log archive
Specific line ka permalink	Line number pe click → URL copy
Search within logs	Log area pe click → Ctrl+F
Timestamps	Gear icon → Show timestamps
Raw text view	Gear icon → View raw logs

Failed step khud expand ho jaata hai — page open karte hi seedha error dikhta hai.

gh CLI (best for daily use)

Command	Kya karta hai
<code>gh run list</code>	Recent runs ki list
<code>gh run view <ID> --log</code>	Poora log terminal me
<code>gh run view <ID> --log-failed</code>	Sirf failed steps ka log
<code>gh run watch</code>	Live streaming
<code>gh run rerun <ID> --failed</code>	Sirf failed jobs rerun
<code>gh run download <ID></code>	Artifacts download
<code>gh workflow run <file>.yaml</code>	Manual trigger

```
# Daily pattern
git push
gh run watch           # live dekho
gh run view --log-failed # fail hua to sirf error
```

Debug logging

Repo → Settings → Secrets and variables → Actions → **Variables** tab:

Variable	Value	Kya deta hai
<code>ACTIONS_STEP_DEBUG</code>	<code>true</code>	Har step ka internal detail
<code>ACTIONS_RUNNER_DEBUG</code>	<code>true</code>	Runner / VM setup detail

Permanently on mat chhodo — logs bloat hote hain.

Job Summary (underrated)

```
- name: Test summary
  if: always()
  run: |
    echo "## Selenium Results" >> $GITHUB_STEP_SUMMARY
    echo "| Metric | Value |" >> $GITHUB_STEP_SUMMARY
    echo "|---|---|" >> $GITHUB_STEP_SUMMARY
    echo "| Passed | $(jq '.summary.passed // 0' reports/report.json) |" >> $GITHUB_STEP_SUMMARY
    echo "| Failed | $(jq '.summary.failed // 0' reports/report.json) |" >> $GITHUB_STEP_SUMMARY
```

Markdown likho, run page pe sundar formatted table dikhega — logs khodne ki zarurat nahi.

Selenium: screenshot on failure

```
# conftest.py
@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        driver = item.funcargs.get("driver")
        if driver:
            Path("reports/screenshots").mkdir(parents=True, exist_ok=True)
            driver.save_screenshot(f"reports/screenshots/{item.name}.png")
```

Headless CI me "locally to chal raha tha" wali problem ka 90% solution yahi hai.

Retention

Cheez	Kitne din
Job logs	90 din (configurable)
Artifacts	Default 90 din, <code>retention-days</code> : se change
S3 pe bheja hua	Tumhare control me — long-term source of truth

12. Locally chalana

VS Code me "Run" button nahi hai — Actions GitHub ke servers pe chalta hai. 3 options:

Option 1: `act` (Docker simulation)

```
# Docker Desktop chahiye
winget install nektos.act
# ya WSL2:
curl -s https://raw.githubusercontent.com/nektos/act/master/install.sh | sudo bash

act -l           # jobs list
act             # default push event
act -j test      # specific job
act workflow_dispatch # manual trigger simulate
act -n           # dry run
act -s APP_PASSWORD=xxx # secret pass karo
act --secret-file .secrets
```

`act` ek `ACT` env variable set karta hai — usse steps skip karo:

```
- name: Upload to S3
  if: ${!env.ACT} # act me skip, GitHub pe chalega
  run: aws s3 cp ...
```

Kaam karta hai	Nahi karta
<code>run:</code> commands	Windows / macOS runners
Common actions (checkout, setup-python)	Artifacts upload (properly)
Matrix strategy	OIDC / AWS role assume
Secrets (manually pass karo)	Caching (partial)
Job dependencies (<code>needs</code>)	GitHub API-dependent actions

Image choose karte time **medium (~500MB)** lo. Micro me Python/Chrome nahi hota; full ~18GB hai.

Option 2: `gh` CLI se remote trigger (recommended)

```
gh workflow run pytest-selenium.yml
gh run watch
gh run view --log-failed
```

Local run nahi hai, par terminal chhodne ki zarurat nahi — aur environment bilkul real hota hai.

Option 3: Steps manually chalao (fastest)

```
pip install -r requirements.txt
mkdir -p reports
HEADLESS=true pytest tests/ --json-report --json-report-file=reports/report.json -v
```

90% waqt tumhe workflow test nahi karna hota — **tests** test karne hote hain.

VS Code setup

Extension: **"GitHub Actions"** (Microsoft official) — YAML autocomplete, live validation, sidebar me runs aur logs, deprecated action version warnings. Run nahi karta, par 80% errors push se pehle pakad leta hai.

Situation	Tool
Test code likh / debug kar rahe ho	Terminal me seedha <code>pytest</code>
YAML syntax check	VS Code extension
Workflow structure test	<code>act -n</code>
Simple job locally verify	<code>act -j test</code>
Selenium + AWS + real secrets	<code>gh workflow run</code>
Fail hua, debug karna hai	<code>gh run view --log-failed</code>

13. Reusable Workflows

Enterprise setups me common. Job level pe `uses:` — steps ki jagah poora job dusri repo se aata hai.

```
# CALLER (tumhari repo)
name: "PR Mergeability Check"
on:
  workflow_dispatch:
    inputs:
      pull_number:
        description: 'PR number to evaluate'
        required: true
        type: string

permissions:
  id-token: write      # OIDC / JWT ke liye
  contents: read       # checkout ke liye

jobs:
  verify:
    uses: MyOrg/shared.workflows/.github/workflows/template.yml@prod
    with:
      pull_number: ${ inputs.pull_number }
    secrets: inherit
```

Kahan uses:	Kya hota hai
Step level	Composite action (actions/checkout@v5)
Job level	Reusable workflow — poora job import

Path breakdown:

```
MyOrg / shared.workflows / .github/workflows/template.yml @ prod
org      repo           file path                ref
```

Do cheezein bhoolna mat:

1. **with:** — inputs explicitly pass karo, implicit context inheritance pe bharosa mat karo.
2. **secrets: inherit** — reusable workflow ko by default **koi secret nahi** milta.

Analogy: Central kitchen. Template repo = kitchen (recipes wahan), tumhari file = order slip, **@prod** = "approved menu se do". Platform team ek baar likhti hai, 200 repos wahi use karte hain.

14. Expression Syntax Reference

Kya chahiye	Syntax
Secret	<code>\${{ secrets.NAME }}</code>
Variable	<code>\${{ vars.NAME }}</code>
Env var	<code>\${{ env.NAME }}</code> ya <code>\$NAME</code> shell me
Manual input	<code>\${{ inputs.NAME }}</code>
Matrix value	<code>\${{ matrix.python }}</code>
Step ka output	<code>\${{ steps.<id>.outputs.<key> }}</code>
Job ka output	<code>\${{ needs.<job>.outputs.<key> }}</code>
Branch name	<code>\${{ github.ref_name }}</code>
Full ref	<code>\${{ github.ref }}</code>
Commit SHA	<code>\${{ github.sha }}</code>
Trigger type	<code>\${{ github.event_name }}</code>
Kisne trigger kiya	<code>\${{ github.actor }}</code>
Run number / ID	<code>\${{ github.run_number }}</code> / <code>\${{ github.run_id }}</code>
Repo name	<code>\${{ github.repository }}</code>
Workspace path	<code>\${{ github.workspace }}</code>
Job status	<code>\${{ job.status }}</code>

15. Common Gotchas

Trap	Galat	Sahi
Tabs	Tab character	Sirf spaces (2 recommended)
Version numbers	<code>python-version: 3.10</code> → banega <code>3.1</code>	<code>python-version: "3.10"</code>
Multi-line command	<code>run: cmd1 && cmd2 && cmd3</code>	<code>run: </code> + indented lines
Checkout bhool gaye	Seedha <code>pip install -r requirements.txt</code>	Pehle <code>actions/checkout@v5</code>
<code>export</code> se variable	<code>run: export VAR=x</code>	<code>echo "VAR=x" >> \$GITHUB_ENV</code>
Report generate nahi hua	<code>run: pytest</code>	<code>pytest --json-report --json-report-file=...</code>
Timeout	Kuch nahi set kiya (default 6h)	<code>timeout-minutes: 30</code>
Schedule chal nahi raha	Feature branch pe cron	Default branch pe merge karo

Debug checklist

Problem	Fix
Workflow trigger hi nahi ho raha	YAML indentation, aur file default branch pe hai?
Locally pass, CI me fail	Headless mode, screen size, missing env var
Detailed logs chahiye	<code>ACTIONS_STEP_DEBUG = true</code>
YAML syntax doubt	VS Code "GitHub Actions" extension

```
# Temporary debug step
- name: Debug
  if: failure()
  run: |
    pwd && ls -la
    env | grep -v -i secret
    cat reports/*.json || echo "no report generated"
```

Build order (recommended)

- 1. Khaali workflow: sirf `workflow_dispatch` + `echo hello`
- 2. Manually run karo — green hona chahiye
- 3. `checkout` + Python setup add karo
- 4. Dependencies install add karo
- 5. Ek single test chalao
- 6. Poora suite chalao
- 7. Artifact upload add karo
- 8. AWS + S3 add karo
- 9. **Ab** `push` aur `schedule` triggers on karo

Ek saath pura likhkar push karne ka temptation hoga — mat karo. 40-line workflow fail hone par pata nahi chalega kaunsi line ne toda.

16. Complete Working Example

`.github/workflows/pytest-selenium.yml`

```
name: Pytest Selenium Suite

on:
  push:
    branches: [main, develop]
  pull_request:
  schedule:
    - cron: "30 2 * * *"      # 02:30 UTC = 08:00 AM IST
  workflow_dispatch:

permissions:
  id-token: write            # OIDC ke liye
  contents: read

env:
  AWS_REGION: us-east-1
  S3_BUCKET: my-test-reports-bucket
  REPORT_FILE: reports/report.json

jobs:
```

```

selenium-tests:
  runs-on: ubuntu-latest
  timeout-minutes: 45

  steps:
    - name: Checkout code
      uses: actions/checkout@v5

    - name: Setup Python
      uses: actions/setup-python@v5
      with:
        python-version: "3.11"
        cache: pip

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt
        pip install pytest-json-report

    - name: Verify Chrome
      run: google-chrome --version

    - name: Run pytest selenium tests
      run: |
        mkdir -p reports
        pytest tests/ \
          --json-report \
          --json-report-file=${{ env.REPORT_FILE }} \
          --junitxml=reports/junit.xml \
          -v
      env:
        HEADLESS: "true"
        BASE_URL: ${vars.BASE_URL}
        APP_PASSWORD: ${secrets.APP_PASSWORD}

    - name: Upload report as artifact
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: test-report-${{ github.run_number }}
        path: reports/
        retention-days: 30

    - name: Configure AWS credentials
      if: always()
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: arn:aws:iam::123456789012:role/github-actions-s3
        aws-region: ${env.AWS_REGION}
        role-session-name: gh-pytest-selenium

    - name: Upload JSON report to S3
      if: always() && github.event_name == 'schedule'
      run: |
        RUN_DATE=$(date -u +'%Y/%m/%d')
        aws s3 cp ${env.REPORT_FILE} \
          s3://${env.S3_BUCKET}/selenium-reports/${RUN_DATE}/report-${{ github.run_id }}.json

    - name: Job summary
      if: always()
      run: |
        echo "### Selenium Run #${{ github.run_number }}" >> $GITHUB_STEP_SUMMARY
        echo "- Trigger: ${github.event_name}" >> $GITHUB_STEP_SUMMARY
        echo "- Branch: ${github.ref_name}" >> $GITHUB_STEP_SUMMARY
        echo "- Status: ${job.status}" >> $GITHUB_STEP_SUMMARY

```

Setup checklist

1. `.github/workflows/` folder banao, YAML file daalo
2. Settings → Secrets and variables me secrets add karo
3. Bucket name, region, role ARN, branch names apne values se replace karo
4. Pehle `workflow_dispatch` se manually chalao
5. Green hone ke baad hi cron pe bharosa karo

Note: Action versions (`@v4` , `@v5`) time ke saath badalte rehte hain. Pehli run me deprecation warning aaye to GitHub Marketplace pe latest check kar lena.